AGENTIC OS
STARTER KIT

Build your own agentic operating system inside Claude Code, from the core out, the way it actually survives.

Two master prompts · the rot model · a worked blueprint · the 5-layer framework

Companion to the video: **Build Your Agentic OS**

by Mark Kashef · skool.com/earlyaidopters

THE MENTAL MODEL · FIVE LAYERS ON A SUBSTRATE

Picture your OS as the Earth: five layers from a solid core out to a changing atmosphere, all sitting on a substrate where memory lives. The one rule that governs everything: **grow inside-out**. Start at the core, who this OS is, and only add an outer layer once real usage has earned it. The deeper the layer, the slower it should ever change.

Layer	What it is	Rots in
1. Identity (CLAUDE.md)	Your point of view, voice, who you serve, defaults, refusals. The core.	Months
2. Rules & Hooks	Black-and-white always / never lists, plus reflexes that fire automatically.	Weeks
3. Skills	Verbs you have done by hand enough to earn a name (about 3 repeats).	Days to weeks
4. Agents	Roles that hold judgment and orchestrate skills. A verb vs a person with a job.	Days
5. Tools, MCPs, CLIs	The wires out to your real software. Wrap in skills, never marry.	Hours
Substrate (.wiki)	Compounding memory under all five layers. Re-ingest, do not rebuild.	Grows

How to use this kit. Pick a prompt below, open a NEW Claude Code session in an empty folder, paste it, and answer the interview. It writes a single **os-blueprint.md** from your own answers, then either walks you through the plan or scaffolds a real starter OS. Prompt A is the cheap, single-threaded build. Prompt B uses dynamic workflows to scan, verify, and maintain on its own.

PROMPT A · THE SIMPLE BUILD

When to use it

You are new to this, or you want it cheap, fast, and predictable. Single-threaded, no agent teams, light on tokens. The fastest path from an empty folder to a working personal OS.

The prompt (copy-paste ready version is in master_prompt_A_simple.txt)

```
You are my agentic OS architect. I want to set up a simple, personal agentic operating system inside
Claude Code, in this folder, and I want you to walk me through it start to finish. Read all of these
instructions before you reply.

The one rule that governs everything: we grow this inside-out. We do not design a big system up front and
hope it holds. We start at the core, which is who this OS is, and we only add an outer layer once it has
been earned by real usage. The deeper the layer, the slower it should ever change. Everything we build is
plain files and folders, so it stays portable and my data stays in readable local files, never trapped
inside one vendor.

Important: keep this SIMPLE and cheap. Do NOT spin up teams of agents or dynamic workflows for this
setup. Do it as a straightforward, single-threaded, guided build. This version is meant to be fast,
predictable, and light on tokens. I have a separate, more powerful version that uses dynamic workflows
for when I want my machine scanned, the build cross-checked, and the system maintained automatically.

Here is the model I want you to teach me and build against. Picture the system as the Earth, five layers
from a solid core out to a changing atmosphere, all of it sitting on a substrate where memory lives.

1. Identity, the core. A CLAUDE.md file that holds my point of view, my voice, who I serve, my defaults,
and the things I refuse to do. This is what makes the OS act like me and not a generic assistant. It
changes the slowest, over months.
2. Rules and Hooks. Rules are black and white constraints that need no judgment, my always-do and
never-do list. Hooks are reflexes that fire automatically at certain moments, like when I start a session
or right before I save a change to a file. Changes over weeks.
3. Skills, the verbs. Workflows I have done by hand enough times that they have crystallized into a named
command. A skill earns its name after roughly three repeats. Changes over days to weeks.
4. Agents and Subagents, the roles. A role that holds judgment and orchestrates several skills underneath
it. A skill is a verb, an agent is a person with a job, like a CFO or an editor. Changes over days.
5. Tools, MCPs, and CLIs, the wires out to my real software. Modular, slot in and out, and they break the
fastest. We wrap them in skills, never marry them. Heads up, this layer usually needs real technical
setup like API keys and configs, often a developer's job, so by default we write down what each
connection would be and leave the live hookup as a clearly marked next step.

Under all five sits the substrate, a compounding .wiki folder. Instead of re-reading my raw documents on
every single query, you maintain a living wiki of them, cross-linked markdown pages that get richer over
time. Three parts: raw sources that you read but never edit, the wiki pages that you write and maintain,
and the schema, which is the CLAUDE.md that tells you how to ingest, query, and tidy the wiki. It starts
empty, so part of your job is to tell me in plain language how to get my real documents into it.

Guardrails the whole way: confirm before anything destructive, before overwriting a file that already
exists, and before acting outside this folder. Giving a role a key is not the same as it needing one, so
scope every connection to the narrowest thing it must reach. Keep client names, secrets, and private data
out of anything that could go public, and flag it if you see it heading there.

STEP 0, CHECK WHERE WE ARE. Before anything else, tell me the exact folder path you are in, list what is
already inside it, and ask me plainly to confirm this is the empty folder where my OS should live. Do not
write or change a single file until I confirm. If it already has things in it, point that out and ask
whether this is a fresh home or an existing project.

STEP 1, INTERVIEW ME. Ask the questions below in small batches of two or three and wait for my answers,
do not dump them all at once. After the basics, dig one level deeper into how I actually work week to
week, because the obvious answer is rarely the real one. One concrete follow up per area, not an
```

```
interrogation. The questions, in order:
1. What is your role, and the business or practice you run or work inside? Explain it like you would to a smart friend who does not know your industry.
2. Who do you serve, and what do they pay you for?
3. What is your point of view in this work, the take or standard that makes you you, and what do you flat out refuse to do?
4. Walk me through a normal week. What recurs, roughly when, and which parts drain you most?
5. What software and tools do you actually pay for and live in day to day, including the boring ones like your accounting tool and your inbox?
6. Where does your important information live right now, and in what form, like folders of PDFs, transcripts, a database, email threads?
7. What do you find yourself re-explaining or re-deciding over and over?
8. What few tasks, if this OS owned them reliably, would change your week the most?
9. Is any of your data sensitive or client confidential, anything that must never leave your machine?
10. Do you mostly want a thinking partner that drafts and analyzes, or a workforce that quietly does recurring jobs, or both, and in what mix?
11. How comfortable are you with technical setup like API keys and config files, and do you have a developer to lean on, or is it just you? Your answer changes how much I build live versus hand you as a clearly marked next step.
When the interview is done, write everything you learned into a single markdown file in this folder called os-blueprint.md, with a plain readable section per layer plus the substrate, and what to build first versus later. Show me that file, play my operation back to me in one short paragraph, and ask me to confirm or correct it before anything is built. This file is the single source of truth.

STEP 2, OFFER ME TWO PATHS. After I confirm, lay these out in plain language and ask me to pick. Steering line first: if you are not comfortable with technical setup like API keys and config files, start with the Blueprint, it is a plan you or a developer act on later and it changes nothing on your machine.
PATH A, Blueprint. Build no files. The os-blueprint.md you wrote IS your blueprint. Walk me through it section by section, tell me for each layer what goes there and what to build first versus later, and stop. Tell me the exact path to the file and that a .md opens in any text editor.
PATH B, Build it out. Scaffold and populate a working starter OS directly, layer by layer, growing from the core out, single-threaded, no agent teams. Go one layer at a time: a plain sentence on what that layer is and why it comes now, then ask only that layer's extra questions, show me what you are about to create, and on my okay write the real files seeded with my answers. Build toward a CLAUDE.md at the root for identity, a .claude/rules/ folder with always.md and never.md, a .claude/skills/ folder with one or two skills I have genuinely earned, a .claude/agents/ folder for a role I already spawn repeatedly, a .claude/hooks/ folder for the reflexes I want, and a .wiki/ folder for the substrate with schema.md, index.md, log.md, and the entities, concepts, and sources folders. Do not invent skills or agents I have not earned, if something is a future item it stays a note in the blueprint, not a file. For Rules and Hooks, give me a plain-English menu of three or four example reflexes to pick from rather than asking an open-ended question, and be honest that turning a hook on may need a one-time config. For Tools, say up front this layer usually needs a developer, and default to documenting each connection with the narrow scope it needs rather than wiring it live. For the .wiki, do not leave me with empty folders, tell me the exact path to drop my real documents and offer to ingest one with me so I see it work.
When you finish, write a README.md with what is here, what is seeded versus still empty, what was left as a next step, and a plain "How to use this tomorrow" section for someone who is not a coder.

Whichever path I pick, narrate what you are doing in plain English, and before each layer give me a sentence or two on what it is and why it comes when it does, so I am choosing with understanding and not just nodding along.

Now begin with Step 0.
```

What good looks like

- An os-blueprint.md written from your own answers, that you confirm before anything is built.
- Two clear paths: a Blueprint you (or a developer) act on later, or a scaffolded starter OS.
- A real core: CLAUDE.md, rules (always / never), one or two earned skills, a role, a hooks menu, a .wiki.
- Nothing destructive without a confirm, and nothing invented that you have not earned.

PROMPT B · THE WORKFLOW BUILD

When to use it

You already have a pile of files and you want the system scanned, the build cross-checked, and the whole thing maintained automatically. Uses dynamic workflows, so it spends more tokens but does far more.

The prompt (copy-paste ready version is in master_prompt_B_workflows.txt)

```
You are my agentic OS architect, and we are going to set up my agentic operating system inside Claude
Code using DYNAMIC WORKFLOWS to do the heavy lifting. Read all of these instructions before you reply.

The one rule that governs everything: we grow this inside-out. We start at the core, which is who this OS
is, and we only add an outer layer once it has been earned by real usage. The deeper the layer, the
slower it should ever change. Everything we build is plain files and folders, so it stays portable and my
data stays in readable local files, never trapped inside one vendor.

How I want you to work, and this part matters. You have dynamic workflows available. The word "workflow"
in a prompt makes Claude write a small custom harness that spawns a team of subagents, each with its own
clean context window, that can verify each other. Use a workflow for the three heavy, wide, or
cross-checked steps below: scanning my existing files, building the OS, and maintaining it. Do NOT use a
workflow to interview me, the interview is a normal back and forth. For everything small and single-file,
just do it directly.

Two things to do well, because they are the real reasons to reach for workflows:
- Give each subagent its OWN focused goal and clean context so it does not drift, get lazy and stop
halfway, or prefer its own unchecked output. When a subagent has a lot to read, run it in its own
worktree and tell it to keep its working set small so it does not blow past its context window and start
working in a degraded state.
- For anything I will rely on, one agent produces a result and a SEPARATE agent adversarially verifies it
against the blueprint before it survives. Treat that second pass as mandatory.

Cost control, up front and honest. Workflows use more tokens than a single pass, so we keep them tight.
Before any workflow, tell me roughly how heavy it will be and put an explicit token budget on it, for
example "use 10k tokens," and prefer scanning a slice first over my whole disk. The build runs once.
Maintenance runs weekly, not hourly. The rest of the time I drive this cheaply with no workflows at all.

Here is the model. Picture the system as the Earth, five layers from a solid core out to a changing
atmosphere, on a substrate where memory lives.
1. Identity, the core. A CLAUDE.md with my point of view, my voice, who I serve, my defaults, and what I
refuse to do. Changes slowest, over months.
2. Rules and Hooks. Rules are black and white constraints that need no judgment. Hooks are reflexes that
fire automatically, like at session start or before a file change. Changes over weeks.
3. Skills, the verbs. Workflows I have done by hand enough times that they earn a named command, after
roughly three repeats. Changes over days to weeks.
4. Agents and Subagents, the roles. A role that holds judgment and orchestrates skills. A skill is a
verb, an agent is a person with a job. Changes over days.
5. Tools, MCPs, and CLIs, the wires out to my real software. Modular, break fastest, wrapped in skills,
never married. Usually needs real technical setup like API keys, often a developer's job, so by default
we document what each connection would be with the narrow scope it needs and leave the live hookup as a
clearly marked next step.

Under all five sits the substrate, a compounding .wiki folder. Instead of re-reading my raw documents on
every query, you maintain a living wiki of them, cross-linked markdown pages that get richer over time.
Three parts: raw sources you read but never edit, the wiki pages you write and maintain, and the schema,
which is the CLAUDE.md that says how to ingest, query, and tidy. It starts empty, so part of your job is
to tell me in plain language how to get my real documents into it.

Guardrails the whole way: confirm before anything destructive, before overwriting an existing file, and
before acting outside this folder. Scope every connection to the narrowest thing it must reach. Keep
```

```
client names, secrets, and private data out of anything that could go public, and flag it if you see it heading there.

STEP 0, CHECK WHERE WE ARE. Before anything else, tell me the exact folder path you are in, list what is in it, and ask me to confirm this is where my OS should live. Do not write or change a file until I confirm.

STEP 1, UNDERSTAND ME. Two ways, and offer me both.
First, the interview, a normal back and forth. Ask the questions below in small batches of two or three and wait for my answers, and dig one level deeper into how I actually work, one concrete follow up per area:
1. Your role and the business or practice you run or work inside, explained simply. 2. Who you serve and what they pay you for. 3. Your point of view and what you refuse to do. 4. A normal week, what recurs and what drains you. 5. The software you actually pay for and live in. 6. Where your important information lives and in what form. 7. What you re-explain or re-decide over and over. 8. The few tasks that would change your week most. 9. Anything sensitive that must never leave your machine. 10. Thinking partner, background workforce, or both. 11. How comfortable you are with API keys and configs, and whether you have a developer.
Second, the shortcut, if I already have a pile of projects and files: build a READ-ONLY workflow that scans the folders I point you at with a read-only auditor subagent that has only read, search, and list tools and no write or edit, fans out across the folders so each agent reads a slice with a clean context, and drafts my map from what is already there. Read only means it reads and reports and never changes a thing. Put a token budget on it. This way I am correcting a draft instead of answering from a blank page.
Either way, write everything into a single os-blueprint.md in this folder, a plain readable section per layer plus the substrate and what to build first, show it to me, and ask me to confirm or correct before anything is built. That file is the single source of truth.

STEP 2, BUILD IT WITH A WORKFLOW. After I confirm the blueprint, lay out two paths and ask me to pick.
PATH A, Blueprint. The os-blueprint.md IS your plan. Walk me through it and stop, nothing gets scaffolded.
PATH B, Build it out. Build a workflow that reads os-blueprint.md and scaffolds the OS from it, layer by layer, from the core out. Because every answer is already in the file, the workflow runs without stopping to ask me. Each generated file gets a second subagent that adversarially checks it against os-blueprint.md, confirming it reflects what I actually said and sits in the right layer, so I get verified files, not a first draft. Narrate each layer in a sentence before you write it, and show me what you are creating. Build toward CLAUDE.md (identity), .claude/rules/ (always.md, never.md), .claude/skills/ (one or two genuinely earned), .claude/agents/ (a role I already spawn), .claude/hooks/ (the reflexes I want), and .wiki/ (schema.md, index.md, log.md, entities/, concepts/, sources/). Do not invent skills or agents I have not earned. Handle Tools and Hooks the careful way: document Tools connections with narrow scopes rather than wiring live, and give me a plain-English menu of example hooks to pick from. For the .wiki, tell me the exact path to drop my real documents and offer to ingest one with me. Keep a token budget on the build and tell me what it spent.

STEP 3, SET UP MAINTENANCE SO IT DOES NOT ROT. This is the step most people skip and it is why these systems fall apart after a month. Build a SECOND workflow whose only job is upkeep. On demand it scans all five layers plus the wiki and returns one short report: skills I have stopped using, rules that now contradict each other, hooks that no longer fire, wiki pages going stale, and where the live OS has drifted from os-blueprint.md, each with a recommended prune-or-refresh action. Use one verifier subagent per rule and a skeptic-persona subagent to review the findings so it reports high-confidence items, not noise. It only reports and recommends, it never deletes without my okay. Write this into a reusable MAINTENANCE.md. Then tell me, in plain language, how to run it on a schedule, and be honest that it uses tokens so weekly is plenty: type slash schedule to run it weekly on its own even when my computer is off, or slash loop on an interval to tick while I work, or just run it by hand. Recommend weekly. You can also press s in the workflow menu to save this workflow into ~/.claude/workflows or ship it inside a skill so I can reuse it.

When you finish, write a README.md with what is here, what is seeded versus empty, which outer layers were left as next steps, and a plain "How to use this tomorrow" for a non-coder.

Now begin with Step 0.
```

What good looks like

- A read-only auditor that scans your existing folders and drafts your map, so you correct instead of starting blank.

• Every generated file adversarially verified against the blueprint by a separate agent before it survives.

• A reusable MAINTENANCE.md plus a weekly schedule, so the OS keeps itself honest instead of rotting.

• An honest token budget on every workflow, run once to build and weekly to maintain.

THE ANTI-ROT MODEL · WHY MOST OS BUILDS DIE

The most common question about an agentic OS is some version of "how do I stop this going stale and useless after a month?" The honest answer: different layers rot at different rates, like a building. The foundation holds for years. The food in the fridge is gone by the weekend. You do not maintain them on the same schedule, and you do not panic when the fast layer moves, because it is supposed to.

The cadence that fights it

- **Monthly, automatic.** A scheduled job walks every file's Revisit date and interviews you to refresh what is past due.
- **Weekly, light.** A maintenance workflow scans all five layers plus the wiki against your blueprint and reports drift, never deleting.
- **On contact, always.** When you are in a skill and spot a missing edge case, you fix it then. The fast layers get maintained by use.

The full model, with the per-file Revisit convention and the expiry register, is in **ROT_template.md**. A complete worked example, a Fractional CFO OS across all five layers, is in **os-blueprint_EXAMPLE.md**, so you can see exactly what these prompts produce.

BUILD ONE LIKE THIS FROM ZERO

Want to build an agentic OS like this from zero, layer by layer?

Inside the Early AI Dopters community you get the full **living** Claude Code course:

- The five layers taught from the ground up, plus the substrate that makes memory compound.
- A one-hour Agentic OS engineering masterclass that goes far deeper than the video.
- Direct access to me and my team of coaches when you hit a wall building yours.
- A working group of builders shipping their own OS systems for business and personal life.

Join here: https://www.skool.com/earlyaidopters/about